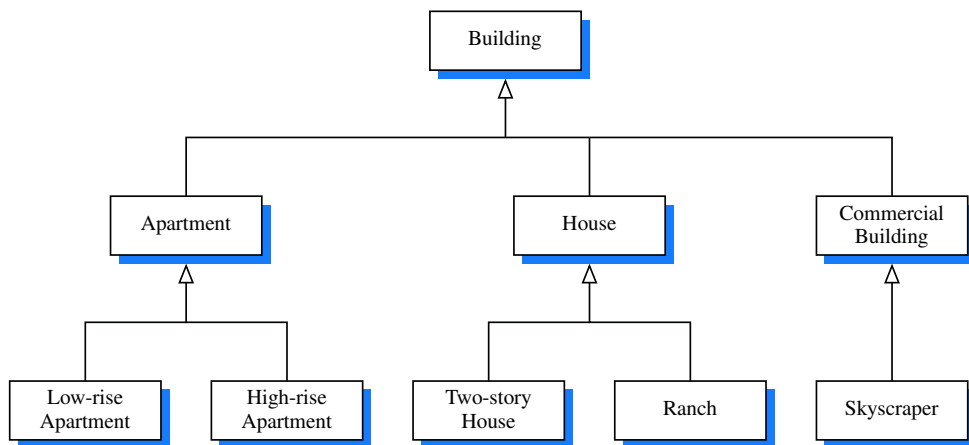


## 2.2 Inheritance

A natural way to organize various structural components of a software package is in a **hierarchical** fashion, with similar abstract definitions grouped together in a level-by-level manner that goes from specific to more general as one traverses up the hierarchy. An example of such a hierarchy is shown in Figure 2.3. Using mathematical notations, the set of houses is a **subset** of the set of buildings, but a **superset** of the set of ranches. The correspondence between levels is often referred to as an “**is a**” **relationship**, as a house is a building, and a ranch is a house.



**Figure 2.3:** An example of an “is a” hierarchy involving architectural buildings.

A hierarchical design is useful in software development, as common functionality can be grouped at the most general level, thereby promoting reuse of code, while differentiated behaviors can be viewed as extensions of the general case. In object-oriented programming, the mechanism for a modular and hierarchical organization is a technique known as **inheritance**. This allows a new class to be defined based upon an existing class as the starting point. In object-oriented terminology, the existing class is typically described as the **base class**, **parent class**, or **superclass**, while the newly defined class is known as the **subclass** or **child class**. We say that the subclass **extends** the superclass.

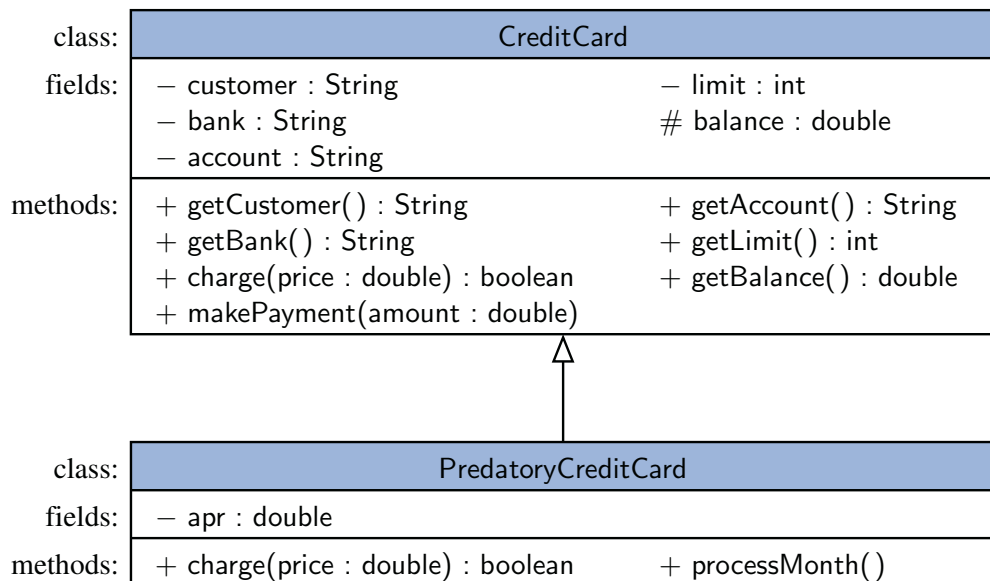
When inheritance is used, the subclass automatically inherits, as its starting point, all methods from the superclass (other than constructors). The subclass can differentiate itself from its superclass in two ways. It may **augment** the superclass by adding new fields and new methods. It may also **specialize** existing behaviors by providing a new implementation that **overrides** an existing method.

### 2.2.1 Extending the CreditCard Class

As an introduction to the use of inheritance, we revisit the `CreditCard` class of Section 1.7, designing a new subclass that, for lack of a better name, we name `PredatoryCreditCard`. The new class will differ from the original in two ways: (1) if an attempted charge is rejected because it would have exceeded the credit limit, a \$5 fee will be charged, and (2) there will be a mechanism for assessing a monthly interest charge on the outstanding balance, using an annual percentage rate (APR) specified as a constructor parameter.

Figure 2.4 provides a UML diagram that serves as an overview of our design for the new `PredatoryCreditCard` class as a subclass of the existing `CreditCard` class. The hollow arrow in that diagram indicates the use of inheritance, with the arrow oriented from the subclass to the superclass.

The `PredatoryCreditCard` class *augments* the original `CreditCard` class, adding a new instance variable named `apr` to store the annual percentage rate, and adding a new method named `processMonth` that will assess interest charges. The new class also *specializes* its superclass by *overriding* the original `charge` method in order to provide a new implementation that assess a \$5 fee for an attempted overcharge.



**Figure 2.4:** A UML diagram showing `PredatoryCreditCard` as a subclass of `CreditCard`. (See Figure 1.5 for the original `CreditCard` design.)

To demonstrate the mechanisms for inheritance in Java, Code Fragment 2.1 presents a complete implementation of the new `PredatoryCreditCard` class. We wish to draw attention to several aspects of the Java implementation.

We begin with the first line of the class definition, which indicates that the new class inherits from the existing `CreditCard` class by using Java's **extends** keyword followed by the name of its superclass. In Java, each class can extend exactly one other class. Because of this property, Java is said to allow only *single inheritance* among classes. We should also note that even if a class definition makes no explicit use of the **extends** clause, it automatically inherits from a class, `java.lang.Object`, which serves as the universal superclass in Java.

We next consider the declaration of the new `apr` instance variable, at line 3 of the code. Each instance of the `PredatoryCreditCard` class will store each of the variables inherited from the `CreditCard` definition (`customer`, `bank`, `account`, `limit`, and `balance`) in addition to the new `apr` variable. Yet we are only responsible for declaring the new instance variable within the subclass definition.

```

1 public class PredatoryCreditCard extends CreditCard {
2     // Additional instance variable
3     private double apr;                                // annual percentage rate
4
5     // Constructor for this class
6     public PredatoryCreditCard(String cust, String bk, String acct, int lim,
7                               double initialBal, double rate) {
8         super(cust, bk, acct, lim, initialBal);        // initialize superclass attributes
9         apr = rate;
10    }
11
12    // A new method for assessing monthly interest charges
13    public void processMonth() {
14        if (balance > 0) { // only charge interest on a positive balance
15            double monthlyFactor = Math.pow(1 + apr, 1.0/12); // compute monthly rate
16            balance *= monthlyFactor;                        // assess interest
17        }
18    }
19
20    // Overriding the charge method defined in the superclass
21    public boolean charge(double price) {
22        boolean isSuccess = super.charge(price);        // call inherited method
23        if (!isSuccess)
24            balance += 5;                                // assess a $5 penalty
25        return isSuccess;
26    }
27 }

```

**Code Fragment 2.1:** A subclass of `CreditCard` that assesses interest and fees.

Constructors are never inherited in Java. Lines 6–10 of Code Fragment 2.1 define a constructor for the new class. When a `PredatoryCreditCard` instance is created, all of its fields must be properly initialized, including any inherited fields. For this reason, the first operation performed within the body of a constructor must be to invoke a constructor of the superclass, which is responsible for properly initializing the fields defined in the superclass.

In Java, a constructor of the superclass is invoked by using the keyword **super** with appropriate parameterization, as demonstrated at line 8 of our implementation:

```
super(cust, mk, acnt, lim, initialBal);
```

This use of the **super** keyword is very similar to use of the keyword **this** when invoking a different constructor within the same class (as described on page 15 of Section 1.2.2). If a constructor for a subclass does not make an explicit call to **super** or **this** as its first command, then an implicit call to **super()**, the zero-parameter version of the superclass constructor, will be made. Returning our attention to the constructor for `PredatoryCreditCard`, after calling the superclass constructor with appropriate parameters, line 9 initializes the newly declared `apr` field. (That field was unknown to the superclass.)

The `processMonth` method is a new behavior, so there is no inherited version upon which to rely. In our model, this method should be invoked by the bank, once each month, to add new interest charges to the customer's balance. From a technical aspect, we note that this method accesses the value of the inherited balance field (at line 14), and potentially modifies that balance at line 16. This is permitted precisely because the balance attributed was declared with **protected** visibility in the original `CreditCard` class. (See Code Fragment 1.5.)

The most challenging aspect in implementing the `processMonth` method is making sure we have working knowledge of how an annual percentage rate translates to a monthly rate. We do not simply divide the annual rate by twelve to get a monthly rate (that would be too predatory, as it would result in a higher APR than advertised). The correct computation is to take the twelfth-root of  $1 + \text{apr}$ , and use that as a multiplicative factor. For example, if the APR is 0.0825 (representing 8.25%), we compute  $\sqrt[12]{1.0825} \approx 1.006628$ , and therefore charge 0.6628% interest per month. In this way, each \$100 of debt will amass \$8.25 of compounded interest in a year. Notice that we use the `Math.pow` method from Java's libraries.

Finally, we consider the new implementation of the `charge` method provided for the `PredatoryCreditCard` class (lines 21–27). This definition *overrides* the inherited method. Yet, our implementation of the new method relies on a call to the inherited method, with syntax **super.charge**(price) at line 22. The return value of that call designates whether the charge was successful. We examine that return value to decide whether to assess a fee, and in either case return that boolean to the caller, so that the new version of `charge` maintains a similar outward interface as the original.

### 2.2.2 Polymorphism and Dynamic Dispatch

The word *polymorphism* literally means “many forms.” In the context of object-oriented design, it refers to the ability of a reference variable to take different forms. Consider, for example, the declaration of a variable having `CreditCard` as its type:

```
CreditCard card;
```

Because this is a reference variable, the statement declares the new variable, which does not yet refer to any card instance. While we have already seen that we can assign it to a newly constructed instance of the `CreditCard` class, Java also allows us to assign that variable to refer to an instance of the `PredatoryCreditCard` subclass. That is, we can do the following:

```
CreditCard card = new PredatoryCreditCard(...); // parameters omitted
```

This is a demonstration of what is known as the *Liskov Substitution Principle*, which states that a variable (or parameter) with a declared type can be assigned an instance from any direct or indirect subclass of that type. Informally, this is a manifestation of the “is a” relationship modeled by inheritance, as a predatory credit card is a credit card (but a credit card is not necessarily predatory).

We say that the variable, `card`, is *polymorphic*; it may take one of many forms, depending on the specific class of the object to which it refers. Because `card` has been declared with type `CreditCard`, that variable may only be used to call methods that are declared as part of the `CreditCard` definition. So we can call `card.makePayment(50)` and `card.charge(100)`, but a compilation error would be reported for the call `card.processMonth()` because a `CreditCard` is not guaranteed to have such a behavior. (That call could be made if the variable were originally declared to have `PredatoryCreditCard` as its type.)

An interesting (and important) issue is how Java handles a call such as `card.charge(100)` when the variable `card` has a declared type of `CreditCard`. Recall that the object referenced by `card` might be an instance of the `CreditCard` class or an instance of the `PredatoryCreditCard` class, and that there are distinct implementations of the `charge` method: `CreditCard.charge` and `PredatoryCreditCard.charge`. Java uses a process known as *dynamic dispatch*, deciding at runtime to call the version of the method that is most specific to the *actual* type of the referenced object (not the declared type). So, if the object is a `PredatoryCreditCard` instance, it will execute the `PredatoryCreditCard.charge` method, even if the reference variable has a declared type of `CreditCard`.

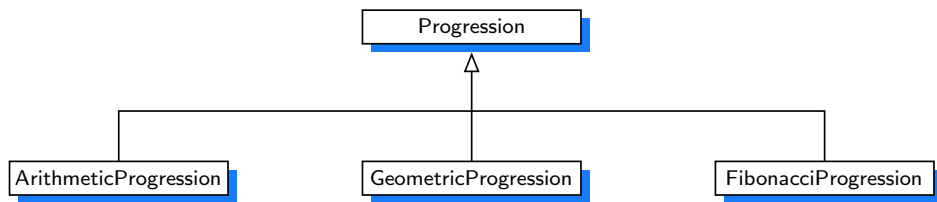
Java also provides an **instanceof** operator that tests, at runtime, whether an instance satisfies as a particular type. For example, the evaluation of the boolean condition, `(card instanceof PredatoryCreditCard)`, produces **true** if the object currently referenced by the variable `card` belongs to the `PredatoryCreditCard` class, or any further subclass of that class. (See Section 2.5.1 for further discussion.)

### 2.2.3 Inheritance Hierarchies

Although a subclass may not inherit from multiple superclasses in Java, a superclass may have many subclasses. In fact, it is quite common in Java to develop complex inheritance hierarchies to maximize the reusability of code.

As a second example of the use of inheritance, we develop a hierarchy of classes for iterating numeric progressions. A numeric progression is a sequence of numbers, where each number depends on one or more of the previous numbers. For example, an *arithmetic progression* determines the next number by adding a fixed constant to the previous value, and a *geometric progression* determines the next number by multiplying the previous value by a fixed constant. In general, a progression requires a first value, and a way of identifying a new value based on one or more previous values.

Our hierarchy stems from a general base class that we name `Progression`. This class produces the progression of whole numbers: 0, 1, 2, .... More importantly, this class has been designed so that it can easily be specialized by other progression types, producing a hierarchy given in Figure 2.5.



**Figure 2.5:** An overview of our hierarchy of progression classes.

Our implementation of the basic `Progression` class is provided in Code Fragment 2.2. This class has a single field, named `current`. It defines two constructors, one accepting an arbitrary starting value for the progression and the other using 0 as the default value. The remainder of the class includes three methods:

`nextValue()`: A public method that returns the next value of the progression, implicitly advancing the value each time.

`advance()`: A protected method that is responsible for advancing the value of `current` in the progression.

`printProgression(n)`: A public utility that advances the progression  $n$  times while displaying each value.

Our decision to factor out the protected `advance()` method, which is called during the execution of `nextValue()`, is to minimize the burden on subclasses, which are solely responsible for overriding the `advance` method to update the `current` field.

```

1  /** Generates a simple progression. By default: 0, 1, 2, ... */
2  public class Progression {
3
4      // instance variable
5      protected long current;
6
7      /** Constructs a progression starting at zero. */
8      public Progression() { this(0); }
9
10     /** Constructs a progression with given start value. */
11     public Progression(long start) { current = start; }
12
13     /** Returns the next value of the progression. */
14     public long nextValue() {
15         long answer = current;
16         advance();    // this protected call is responsible for advancing the current value
17         return answer;
18     }
19
20     /** Advances the current value to the next value of the progression. */
21     protected void advance() {
22         current++;
23     }
24
25     /** Prints the next n values of the progression, separated by spaces. */
26     public void printProgression(int n) {
27         System.out.print(nextValue());           // print first value without leading space
28         for (int j=1; j < n; j++)
29             System.out.print(" " + nextValue()); // print leading space before others
30         System.out.println();                   // end the line
31     }
32 }

```

**Code Fragment 2.2:** General numeric progression class.

The body of the `nextValue` method temporarily records the current value of the progression, which will soon be returned, and then calls the protected `advance` method in order to update the value in preparation for a subsequent call.

The implementation of the `advance` method in our `Progression` class simply increments the current value. This method is the one that will be overridden by our specialized subclasses in order to alter the progression of numbers.

In the remainder of this section, we present three subclasses of the `Progression` class—`ArithmeticProgression`, `GeometricProgression`, and `FibonacciProgression`—which respectively produce arithmetic, geometric, and Fibonacci progressions.

### An Arithmetic Progression Class

Our first example of a specialized progression is an arithmetic progression. While the default progression increases its value by one in each step, an arithmetic progression adds a fixed constant to one term of the progression to produce the next. For example, using an increment of 4 for an arithmetic progression that starts at 0 results in the sequence 0, 4, 8, 12, . . . .

Code Fragment 2.3 presents our implementation of an `ArithmeticProgression` class, which relies on `Progression` as its base class. We include three constructor forms, with the most general (at lines 12–15) accepting an increment value and a start value, such that `ArithmeticProgression(4, 2)` produces the sequence 2, 6, 10, 14, . . . . The body of that constructor invokes the superclass constructor, with syntax **super**(start), to initialize current to the given start value, and then it initializes the increment field introduced by this subclass.

For convenience, we offer two additional constructors, so that the default progression produces 0, 1, 2, 3, . . . , and a one-parameter constructor produces an arithmetic progression with a given increment value (but a default starting value of 0).

Finally (and most importantly), we override the protected advance method so that the given increment is added to each successive value of the progression.

```

1  public class ArithmeticProgression extends Progression {
2
3      protected long increment;
4
5      /** Constructs progression 0, 1, 2, ... */
6      public ArithmeticProgression() { this(1, 0); }      // start at 0 with increment of 1
7
8      /** Constructs progression 0, stepsize, 2*stepsize, ... */
9      public ArithmeticProgression(long stepsize) { this(stepsize, 0); }      // start at 0
10
11     /** Constructs arithmetic progression with arbitrary start and increment. */
12     public ArithmeticProgression(long stepsize, long start) {
13         super(start);
14         increment = stepsize;
15     }
16
17     /** Adds the arithmetic increment to the current value. */
18     protected void advance() {
19         current += increment;
20     }
21 }
```

**Code Fragment 2.3:** Class for arithmetic progressions, which inherits from the general progression class shown in Code Fragment 2.2.



## A Geometric Progression Class

Our second example of a specialized progression is a geometric progression, in which each value is produced by multiplying the preceding value by a fixed constant, known as the *base* of the geometric progression. The starting point of a geometric progression is traditionally 1, rather than 0, because multiplying 0 by any factor results in 0. As an example, a geometric progression with base 2, starting at value 1, produces the sequence 1, 2, 4, 8, 16, ... .

Code Fragment 2.4 presents our implementation of a `GeometricProgression` class. It is quite similar to the `ArithmeticProgression` class in terms of the programming techniques used. In particular, it introduces one new field (the base of the geometric progression), provides three forms of a constructor for convenience, and overrides the protected `advance` method so that the current value of the progression is multiplied by the base at each step.

In the case of a geometric progression, we have chosen to have the default (zero-parameter) constructor use a starting value of 1 and a base of 2 so that it produces the progression 1, 2, 4, 8, ... . The one-parameter version of the constructor accepts an arbitrary base and uses 1 as the starting value, thus `GeometricProgression(3)` produces the sequence 1, 3, 9, 27, ... . Finally, we offer a two-parameter version accepting both a base and start value, such that `GeometricProgression(3, 2)` produces the sequence 2, 6, 18, 54, ... .

```

1  public class GeometricProgression extends Progression {
2
3      protected long base;
4
5      /** Constructs progression 1, 2, 4, 8, 16, ... */
6      public GeometricProgression() { this(2, 1); }           // start at 1 with base of 2
7
8      /** Constructs progression 1, b, b^2, b^3, b^4, ... for base b. */
9      public GeometricProgression(long b) { this(b, 1); }     // start at 1
10
11     /** Constructs geometric progression with arbitrary base and start. */
12     public GeometricProgression(long b, long start) {
13         super(start);
14         base = b;
15     }
16
17     /** Multiplies the current value by the geometric base. */
18     protected void advance() {
19         current *= base;                                     // multiply current by the geometric base
20     }
21 }
```

**Code Fragment 2.4:** Class for geometric progressions.

### A Fibonacci Progression Class

As our final example, we demonstrate how to use our progression framework to produce a ***Fibonacci progression***. Each value of a Fibonacci series is the sum of the two most recent values. To begin the series, the first two values are conventionally 0 and 1, leading to the Fibonacci series 0, 1, 1, 2, 3, 5, 8, . . . . More generally, such a series can be generated from any two starting values. For example, if we start with values 4 and 6, the series proceeds as 4, 6, 10, 16, 26, 42, . . . .

Code Fragment 2.5 presents an implementation of the `FibonacciProgression` class. This class is markedly different from those for the arithmetic and geometric progressions because we cannot determine the next value of a Fibonacci series solely from the current one. We must maintain knowledge of the two most recent values. Our `FibonacciProgression` class introduces a new member, named `prev`, to store the value that preceded the current one (which is stored in the inherited `current` field).

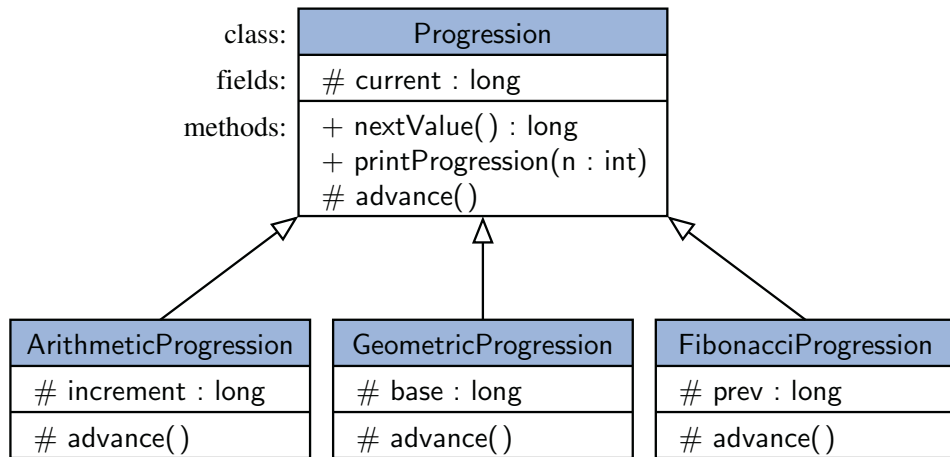
However, the question arises as to how to initialize the previous value in the constructor, when provided with the desired first and second values as parameters. The first should be stored as `current` so that it is reported by the first call to `nextValue()`. Within that method call, an assignment will set the new `current` value (which will be the second value reported) equal to the first value plus the “previous.” By initializing the previous value to  $(\text{second} - \text{first})$ , the initial advancement will set the new `current` value to  $\text{first} + (\text{second} - \text{first}) = \text{second}$ , as desired.

```

1  public class FibonacciProgression extends Progression {
2
3      protected long prev;
4
5      /** Constructs traditional Fibonacci, starting 0, 1, 1, 2, 3, ... */
6      public FibonacciProgression() { this(0, 1); }
7
8      /** Constructs generalized Fibonacci, with give first and second values. */
9      public FibonacciProgression(long first, long second) {
10         super(first);
11         prev = second - first;      // fictitious value preceding the first
12     }
13
14     /** Replaces (prev,current) with (current, current+prev). */
15     protected void advance() {
16         long temp = prev;
17         prev = current;
18         current += temp;
19     }
20 }

```

**Code Fragment 2.5:** Class for the Fibonacci progression.



**Figure 2.6:** Detailed inheritance diagram for class `Progression` and its subclasses.

As a summary, Figure 2.6 presents a more detailed version of our inheritance design than was originally given in Figure 2.5. Notice that each of these classes introduces an additional field that allows it to properly implement the `advance()` method in an appropriate manner for its progression.

### Testing Our Progression Hierarchy

To complete our example, we define a class `TestProgression`, shown in Code Fragment 2.6, which performs a simple test of each of the three classes. In this class, variable `prog` is polymorphic during the execution of the main method, since it references objects of class `ArithmeticProgression`, `GeometricProgression`, and `FibonacciProgression` in turn. When the main method of the `TestProgression` class is invoked by the Java runtime system, the output shown in Code Fragment 2.7 is produced.

The example presented in this section is admittedly simple, but it provides an illustration of an inheritance hierarchy in Java. As an interesting aside, we consider how quickly the numbers grow in the three progressions, and how long it would be before the long integers used for computations overflow. With the default increment of one, an arithmetic progression would not overflow for  $2^{63}$  steps (that is approximately 10 billion billions). In contrast, a geometric progression with base  $b = 3$  will overflow a long integer after 40 iterations, as  $3^{40} > 2^{63}$ . Likewise, the 94th Fibonacci number is greater than  $2^{63}$ ; hence, the Fibonacci progression will overflow a long integer after 94 iterations.

```

1  /** Test program for the progression hierarchy. */
2  public class TestProgression {
3      public static void main(String[] args) {
4          Progression prog;
5          // test ArithmeticProgression
6          System.out.print("Arithmetic progression with default increment: ");
7          prog = new ArithmeticProgression();
8          prog.printProgression(10);
9          System.out.print("Arithmetic progression with increment 5: ");
10         prog = new ArithmeticProgression(5);
11         prog.printProgression(10);
12         System.out.print("Arithmetic progression with start 2: ");
13         prog = new ArithmeticProgression(5, 2);
14         prog.printProgression(10);
15         // test GeometricProgression
16         System.out.print("Geometric progression with default base: ");
17         prog = new GeometricProgression();
18         prog.printProgression(10);
19         System.out.print("Geometric progression with base 3: ");
20         prog = new GeometricProgression(3);
21         prog.printProgression(10);
22         // test FibonacciProgression
23         System.out.print("Fibonacci progression with default start values: ");
24         prog = new FibonacciProgression();
25         prog.printProgression(10);
26         System.out.print("Fibonacci progression with start values 4 and 6: ");
27         prog = new FibonacciProgression(4, 6);
28         prog.printProgression(8);
29     }
30 }

```

**Code Fragment 2.6:** Program for testing the progression classes.

Arithmetic progression with default increment: 0 1 2 3 4 5 6 7 8 9  
 Arithmetic progression with increment 5: 0 5 10 15 20 25 30 35 40 45  
 Arithmetic progression with start 2: 2 7 12 17 22 27 32 37 42 47  
 Geometric progression with default base: 1 2 4 8 16 32 64 128 256 512  
 Geometric progression with base 3: 1 3 9 27 81 243 729 2187 6561 19683  
 Fibonacci progression with default start values: 0 1 1 2 3 5 8 13 21 34  
 Fibonacci progression with start values 4 and 6: 4 6 10 16 26 42 68 110

**Code Fragment 2.7:** Output of the TestProgression program of Code Fragment 2.6.